

Game Engine Concepts

Fachhochschule Oberösterreich

Game Loop and Physics

Game Loop

Introduction

→ What is the central component of a game engine?

→ The **Game Loop**

```
int main()
{
    while (true)
    {
        // Do something
    }
}
```

What is the issue here?
The loop would run as fast as the machine allows, but a central concept in games/simulation is TIME!



Game Loop

Timelines

→ Real-Time

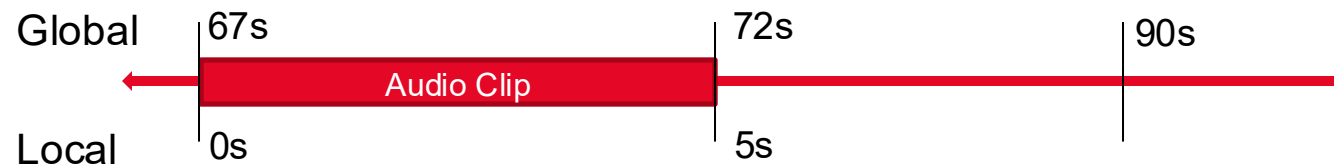
- The time as measured by the CPU high-resolution timer (“real world clock”)
- Not always 100% accurate

→ Game Time

- Game time is technically independent of the real-time (i.e., fast forward, pause, etc.)

→ Global vs. Local time

- Processes in the engine have their own “local” timelines (i.e., audio, animation, etc.)
- Those are “mapped” to the global time



Game Loop

Timelines

- We do not speak of times, but of **rates or frequencies!**
- **Frames Per Second (FPS)**: ~25 for fluid perception, larger is better (we deal with 25 for simplicity)
- Physics update rate: ~120 per second
- AI behaviors: ~1 per second (depending on requirements)

- Early games did not consider time in their update loop (sometimes games were running too fast/slow depending on the machine)
- Typical way: **Measure delta time** since last frame update in game loop
- More sophisticated: **Running average** or **fixed update rate**

Game Loop

Implementation

```
initialize last_time ← current_system_time_in_milliseconds()  
initialize frame_counter ← 0
```

loop forever:

```
    current_time ← current_system_time_in_milliseconds()  
    delta_time_ms ← current_time - last_time  
    delta_time_s ← delta_time_ms / 1000.0
```

```
    if delta_time_s >= (1 / framerate):  
        delta_time_s ← 1 / framerate
```

```
        // Do something
```

```
    last_time ← current_time  
    frame_counter ← frame_counter + 1
```

Game Loop

Implementation

```
initialize last_time ← current_system_time_in_milliseconds()  
initialize frame_counter ← 0
```

loop forever:

```
    current_time ← current_system_time_in_milliseconds()  
    delta_time_ms ← current_time - last_time  
    delta_time_s ← delta_time_ms / 1000.0
```

```
    if delta_time_s >= (1 / framerate):  
        delta_time_s ← 1 / framerate
```

```
        // Do something
```

```
        last_time ← current_time  
        frame_counter ← frame_counter + 1
```

We calculate how many milliseconds passed since the last update and convert the result into seconds (division by 1000.0 of milliseconds)

Game Loop

Implementation

```
initialize last_time ← current_system_time_in_milliseconds()  
initialize frame_counter ← 0
```

loop forever:

```
    current_time ← current_system_time_in_milliseconds()  
    delta_time_ms ← current_time - last_time  
    delta_time_s ← delta_time_ms / 1000.0
```

```
    if delta_time_s >= (1 / framerate):  
        delta_time_s ← 1 / framerate
```

```
        // Do something
```

```
        last_time ← current_time  
        frame_counter ← frame_counter + 1
```

If enough time has passed regarding to the desired framerate ($1/\text{framerate}$), we execute the next iteration of the game loop, otherwise, we “wait” for more delta time

Game Loop

Implementation

```
initialize last_time ← current_system_time_in_milliseconds()  
initialize frame_counter ← 0
```

loop forever:

```
    current_time ← current_system_time_in_milliseconds()  
    delta_time_ms ← current_time - last_time  
    delta_time_s ← delta_time_ms / 1000.0
```

```
    if delta_time_s >= (1 / framerate):  
        delta_time_s ← 1 / framerate
```

```
        // Do something
```

```
    last_time ← current_time  
    frame_counter ← frame_counter + 1
```

We set the deltatime to the desired update rate

Game Loop

Implementation

```
initialize last_time ← current_system_time_in_milliseconds()  
initialize frame_counter ← 0
```

loop forever:

```
    current_time ← current_system_time_in_milliseconds()  
    delta_time_ms ← current_time - last_time  
    delta_time_s ← delta_time_ms / 1000.0
```

```
    if delta_time_s >= (1 / framerate):  
        delta_time_s ← 1 / framerate
```

```
    // Do something
```

```
    last_time ← current_time  
    frame_counter ← frame_counter + 1
```

Finally, we increase the frame counter and reset the timer for the next frame iteration

Game Loop

Implementation

loop forever:

```
current_time ← current_system_time_in_milliseconds()  
delta_time ← current_time - last_time  
delta_time_seconds ← delta_time / 1000.0
```

```
if delta_time_seconds ≥ (1 / framerate):  
    delta_time_seconds ← 1 / framerate
```

```
// Do something
```

```
last_time ← current_time  
frame_counter ← frame_counter + 1  
print(frame_counter)
```

What's next?

Game Loop

Implementation

loop forever:

```
current_time ← current_system_time_in_milliseconds()  
delta_time ← current_time - last_time  
delta_time_seconds ← delta_time / 1000.0
```

```
if delta_time_seconds ≥ (1 / framerate):  
    delta_time_seconds ← 1 / framerate
```

```
// Game loop steps  
CheckUserInput()  
ApplyGamePhysics()  
Render()
```

```
last_time ← current_time  
frame_counter ← frame_counter + 1  
print(frame_counter)
```

We would check if there is some user input
We apply the physical calculations to make the world dynamic
We render the output of the game

(for simplicity, we use the same update rate here)

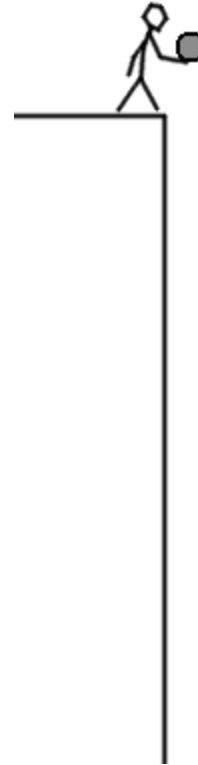
Game Loop

Physics

→ What does this equation calculate?

$$y(t + 1) = v_0 t + y_0 - \frac{1}{2} g t^2$$

→ The height of an **object in free fall** (Newtonian physics)!



Game Loop

Physics

→ An **object in free fall**

- $y(t)$ = altitude at time t
- t = time
- v_0 = initial velocity
- y_0 = initial altitude
- g = gravity (9.81 m/s^2)

→ Let's **simulate an object falling from 100m**

- $y(0) = 100$
- $y(1) = 95,095$
- $y(2) = 80,38$
- $y(3) = 55,62$
- $y(4) = 21,52$
- $y(5) = -22,625$

$$y(t + 1) = v_0 t + y_0 - \frac{1}{2} g t^2$$

$$y(0) = 0 + 100 - 0$$

$$y(1) = 0 + 100 - 4,905$$

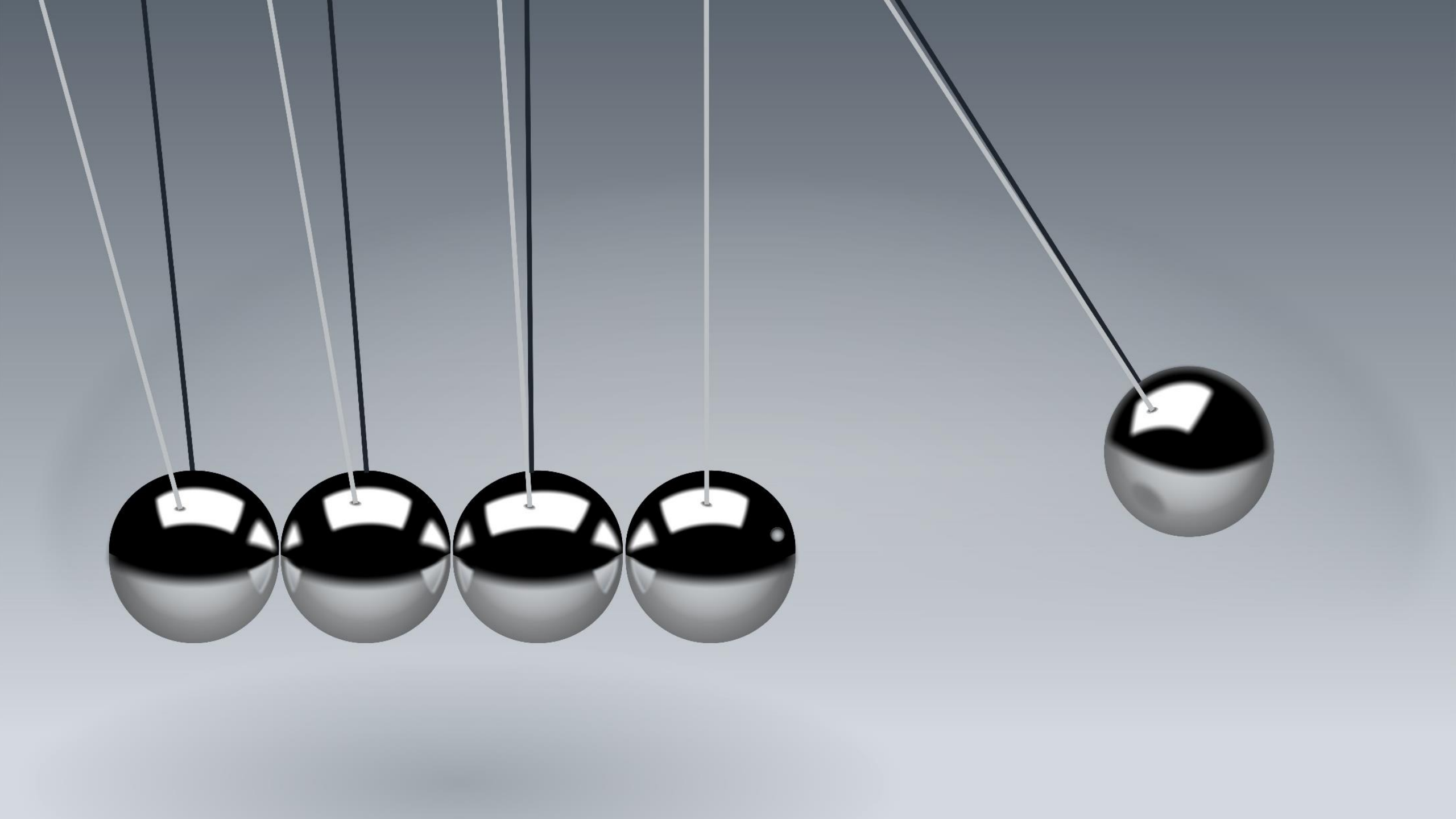
$$y(2) = 0 + 100 - 19,62$$

$$y(3) = 0 + 100 - 44,145$$

$$y(4) = 0 + 100 - 78,48$$

$$y(5) = 0 + 100 - 122,625$$







EA Sports Fifa (1993)



Which aspects of the game may use physics, and what for ...and which do not?

Game Loop

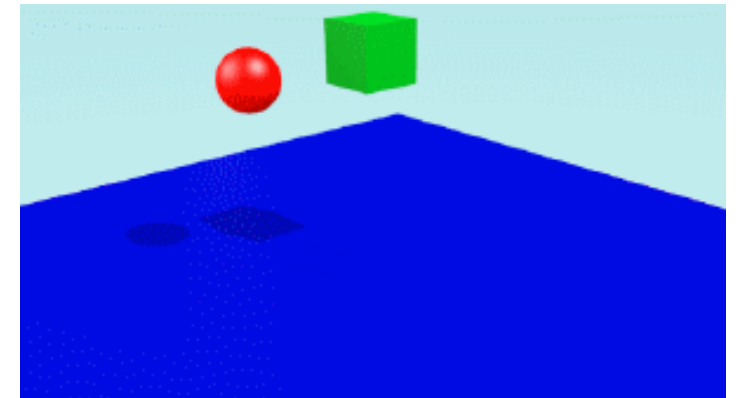
Physics

→ Two major components

- **Collision detection** system
- **Rigid body dynamics** simulation
 - Rigid body: idealized, infinitely hard solid object (i.e., it does not deform) -> does not exist in reality
 - Calculates how rigid bodies move and interact with physical forces

→ Combination of collisions and physics

- Bouncing between objects
- Sliding under friction
- Rolling until stop



Game Loop

Physics

- Do we even need physics?
- Physics can result in chaos
- Physics does not per se guarantee fun

He shoots... he scores! How FIFA 12's physics engine made players snog, levitate and play with broken legs

By [TED THORNHILL](#)

UPDATED: 08:27 BST, 9 December 2011



26
View comments



Zum Beenden des Vollbildmodus Esc drücken

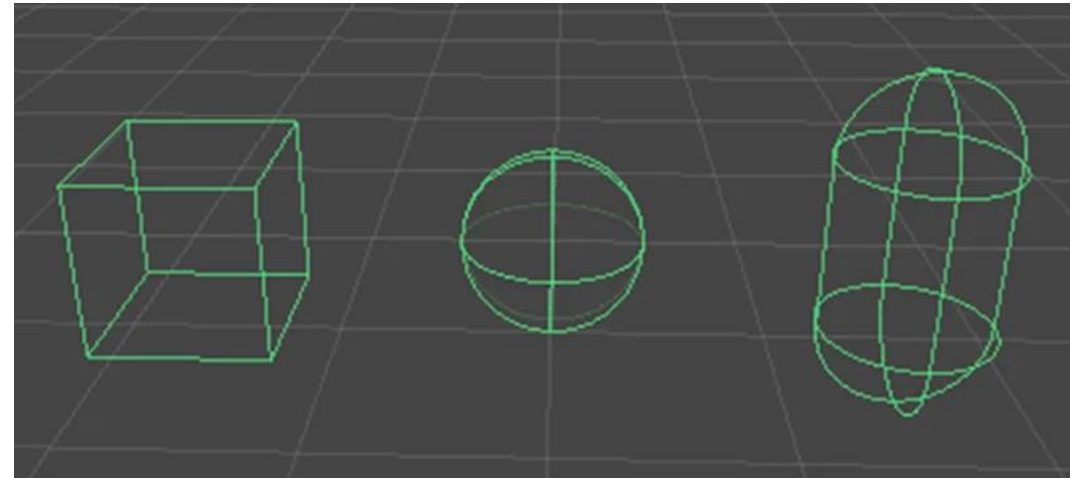


Game Loop

Physics

→ Collision Detection

- Determines when objects in the game world get into contact
- Therefore, each logical object is represented by one or more simple shapes, called **colliders that may or may not intersect**
- **Colliders**
 - Data structure independent from an object's representation in the game
 - **Simplicity is the key:** a stone is a sphere, a car is a box, a human a collection of capsules
 - Are represented with a shape and a transform
 - The shape describes the general form of the collider
 - The transform positions and orients the shape in 3D space

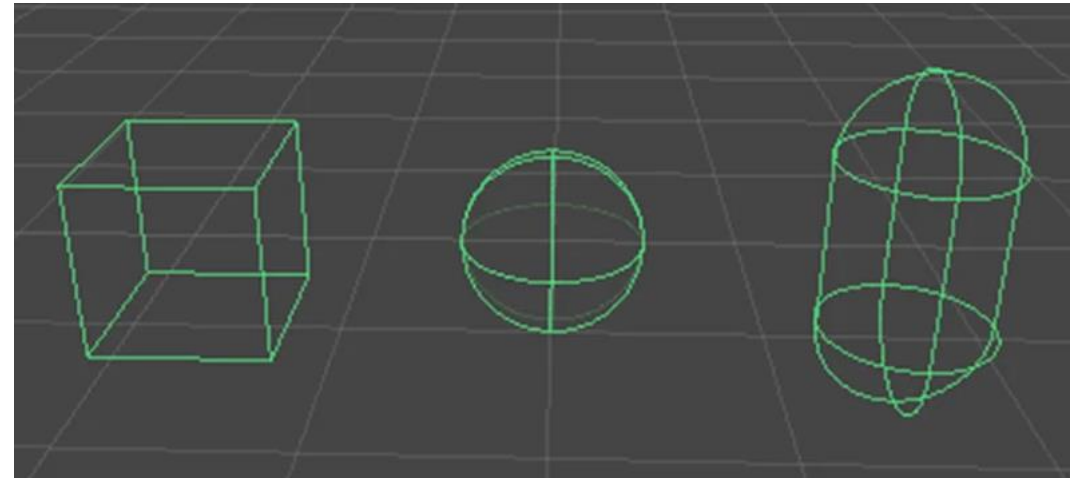


Game Loop

Physics

→ Collision and Physics World

- **Data structure with all colliders**, similar to the scene graph
- Holding the “**collision world**” independently increases performance
 - Only collidable objects are stored
 - Colliders do not need to be indexed via the scene
 - Collisions and physics can be evaluated without graphics
- The same concept applies for the “**physics world**”, which holds rigid bodies
- Today, both are typically strongly tied together

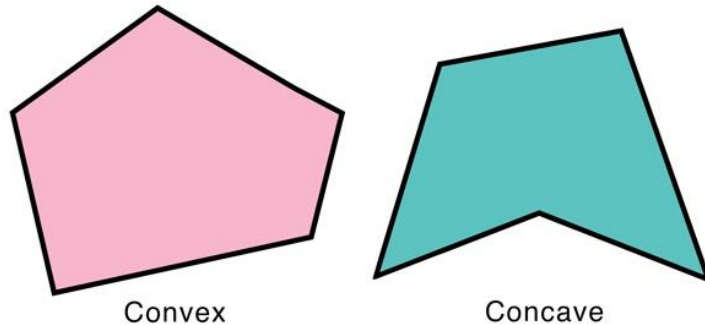


Game Loop

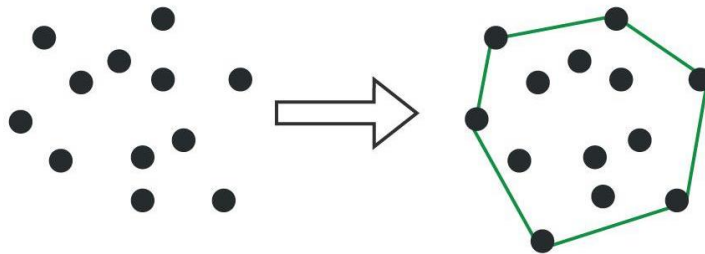
Physics

→ Convex and Concave Shapes

- **Convex shapes:** no ray starting within the shape will pass the surface more than once
- **Concave shapes:** there exist rays within the shape that pass the surface multiple times



- **Convex hull:** smallest convex shape around a concave object

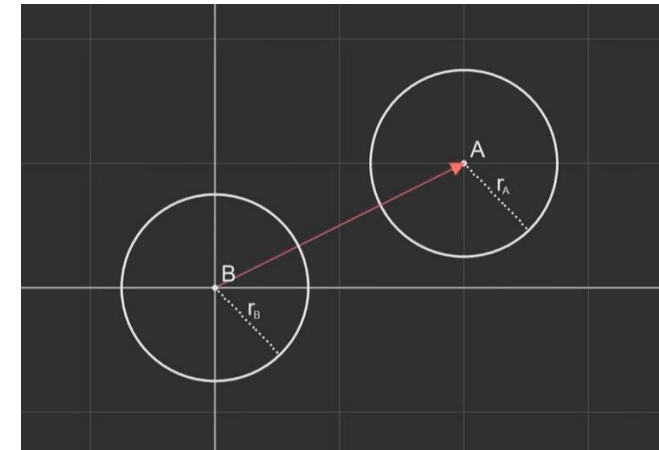
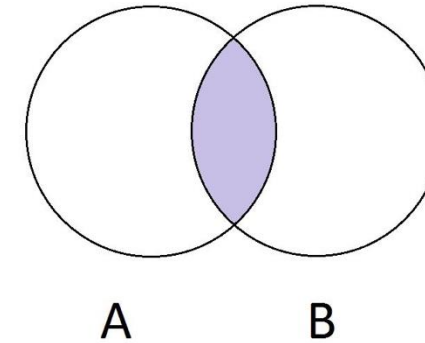


Game Loop

Physics

→ Intersections

- Intersections drive collisions and physics
- **Set-Theory**: the subset of members existing in both sets
- **Geometrical**: the set of all (infinite) points that lie within both shapes
- Point vs. sphere
- Sphere vs. sphere



Game Loop

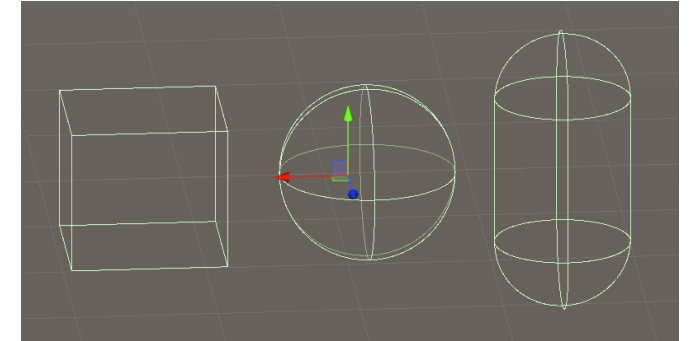
Physics

→ Intersections

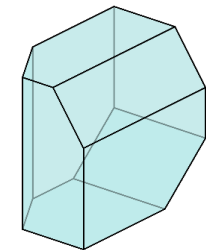
- **Collision primitives**

- Spheres (point + radius)
- Capsule (cylindric, 2 points + radius)
- Box (2 vectors min + max)
- K-Polytopes (dumps planes at normal vectors until roughly fitting an object's shape)
- Convex Volumes (collection of triangles and quads)
- Polytope (arbitrary non-convex shape)

- **Compound:** collection of collision primitives



Box, Sphere, Capsule Collider



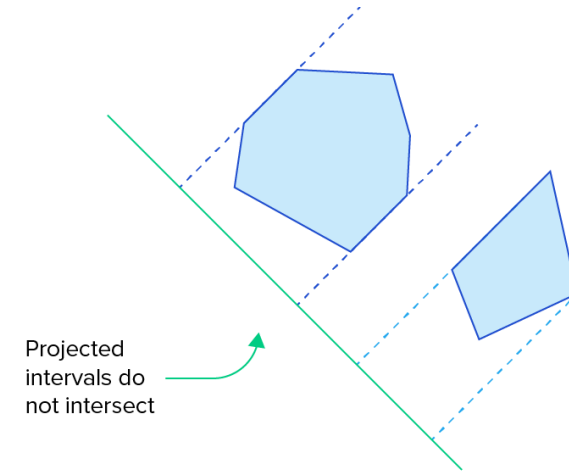
Polytope

Game Loop

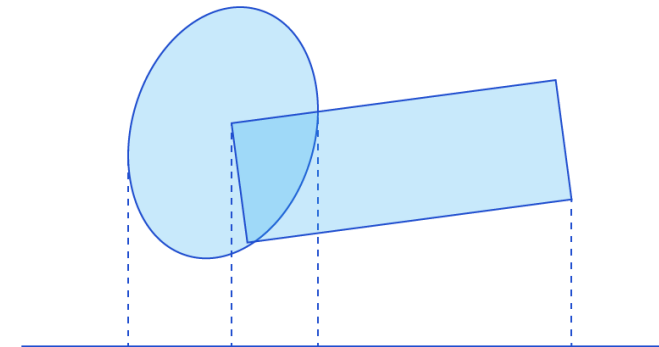
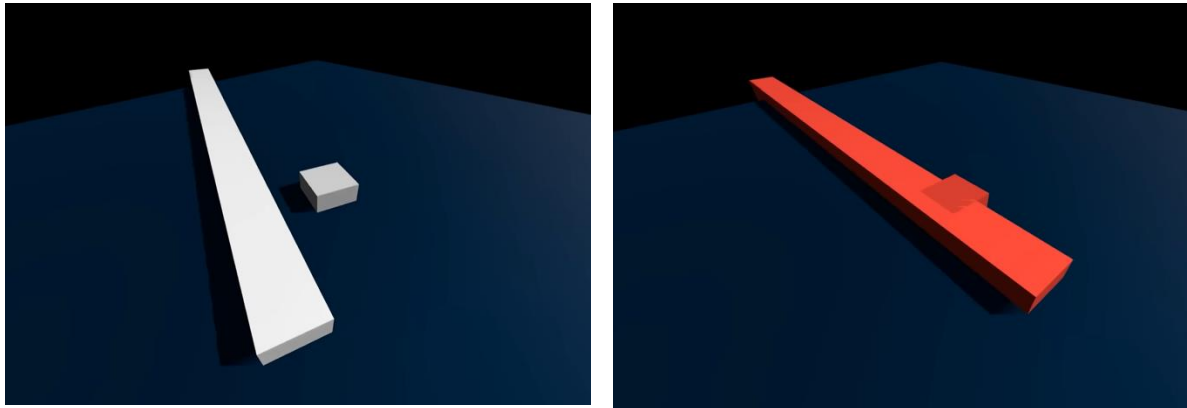
Physics

→ Intersections

- **Separating axis theorem:** calculation of potential overlaps of collider projections on each 3D axis
- If there is an overlap in all axis, we have a collision
- Issue: Concave collisions



Challenge: find a separating axis

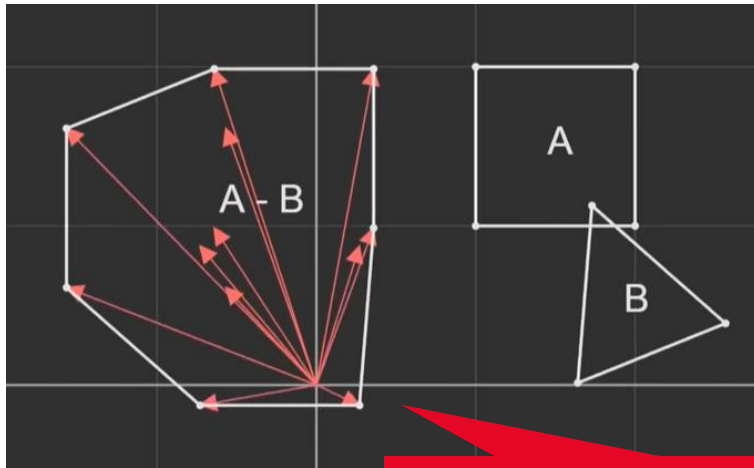


Game Loop

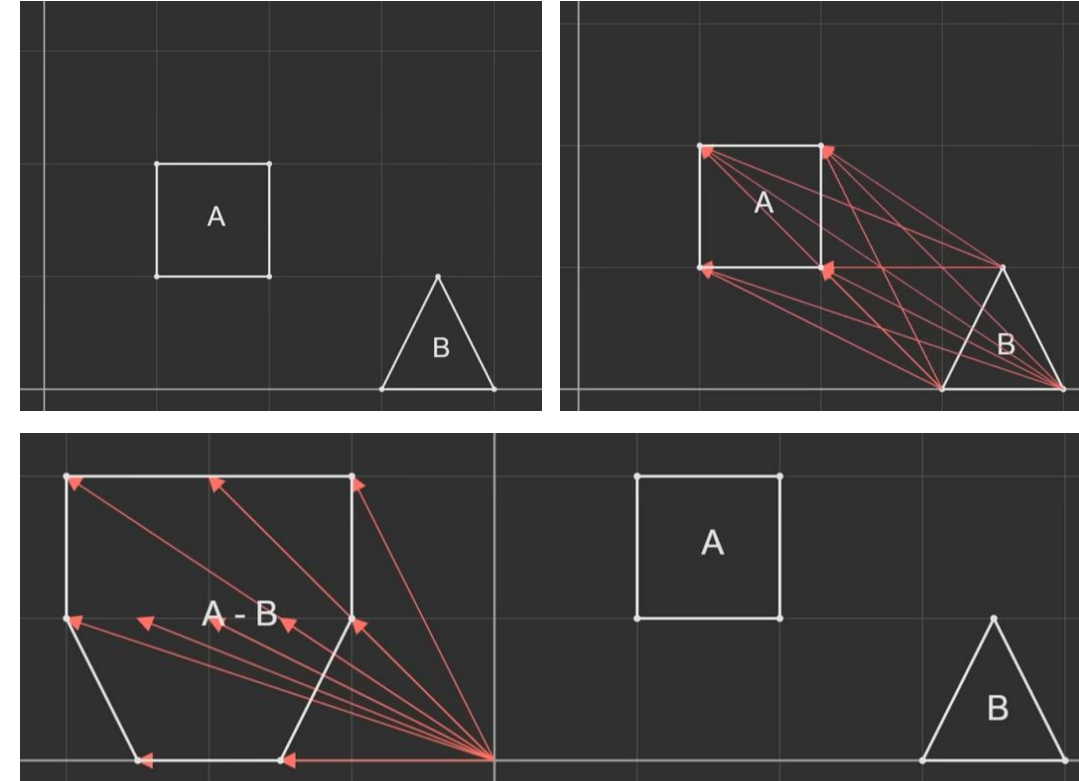
Physics

→ Intersections

- **Minkowski difference:** if the subtraction of all points in object A from all points in object B contains in the diagram origin, they intersect (if a point x is in both objects, $x - x == 0$)
- Can detect concave collisions
- Precision is depending on the number of included points



Collision detected: origin is **inside** the convex hull



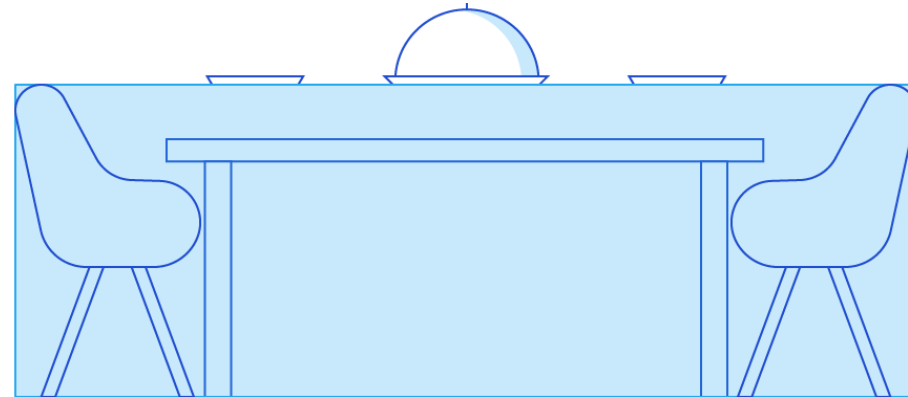
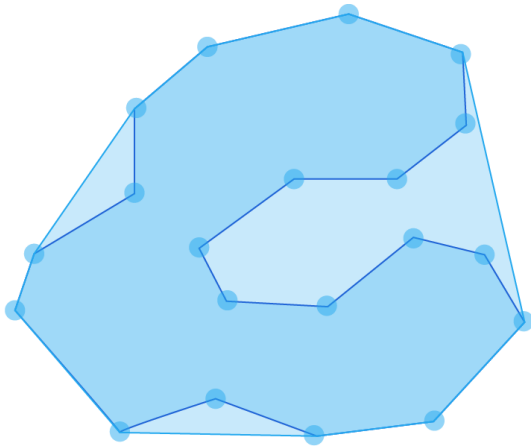
No collision detected: origin is **outside** the convex hull

Game Loop

Physics

→ Intersections

- **Collider Hierarchies:** replace complex combinations with simple primitives and only calculate details when the overarching primitives intersect

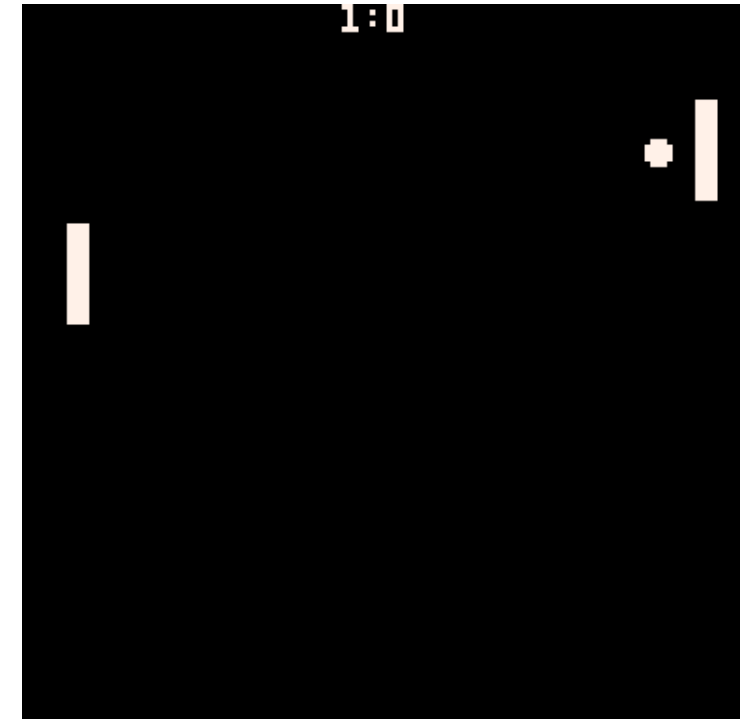


Game Loop

Physics

→ Intersections

- **Bullet-through-paper problem (“tunneling”)**
- If objects are “thin” and speeds are fast, the engine might miss a collision
- **Swept-shapes:** colliders are stretched to represent a “spacetime” collider
- **Continuous Collision Detection:** Does not only calculate collisions frame by frame but also time-to-impact between objects that may collide



Game Loop

Physics

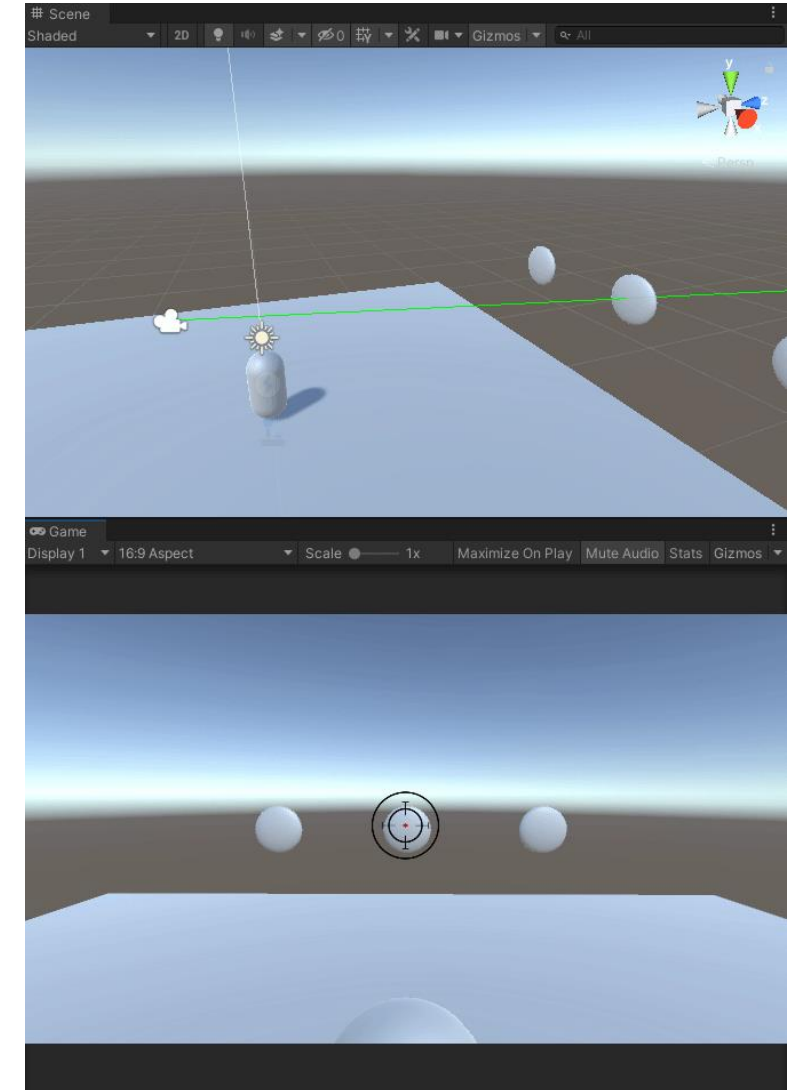
→ Intersections

- **Collision Queries**

- Answer “what if” questions
- **Ray cast:** finding points on a line that intersect with colliders
- **Shape cast:** finding points on a shape-sized volume that intersects with colliders
- **Phantom:** shape cast with zero distance (and “invisible” collider)

- **Collision masks/layers:** categorize which types of objects can collide with other types of objects

- **Collision callbacks:** invoke certain functions when certain collisions (enter, within, exit) are present

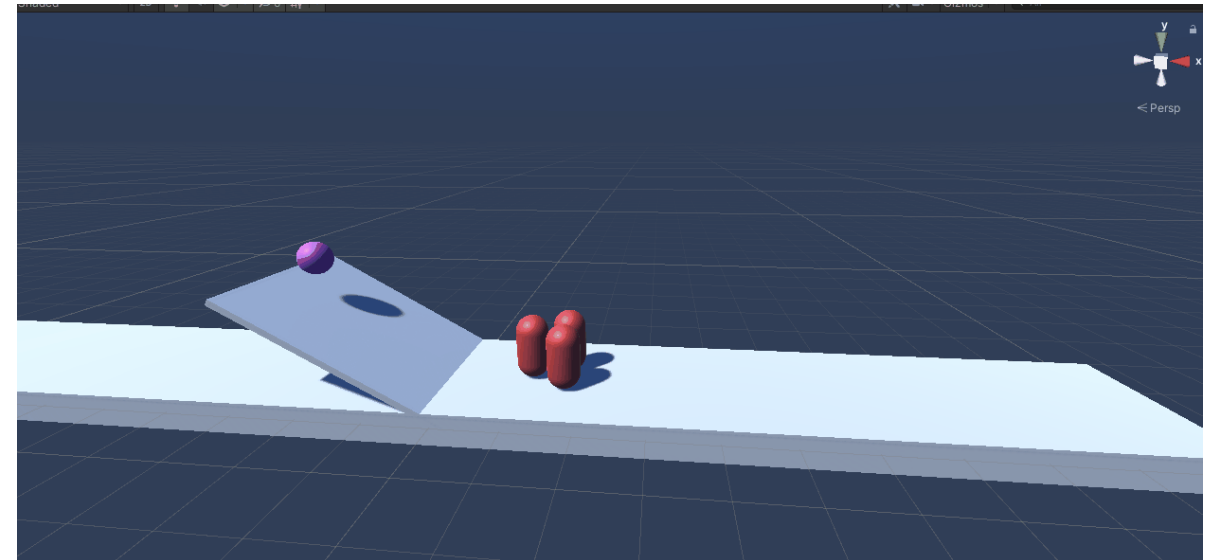


Game Loop

Physics

→ Rigid Body Dynamics

- **Classical rigid body dynamics: Newtonian mechanics**
 - Objects large enough to omit quantum effects
 - Objects slow enough to omit relativistic effects
 - Rigid bodies cannot be deformed and are solid (i.e., constant form)
- **Constraint collisions** forbid objects to overlap and provide immediate response
 - In modern engines, the collision world and physics world are identical
- **At each time step**
 - Potential update transformations are performed
 - Then, the physics component applies additional transformations



Game Loop

Physics

→ Rigid Body Dynamics

- **MKS Units**

- meters, kilograms, seconds

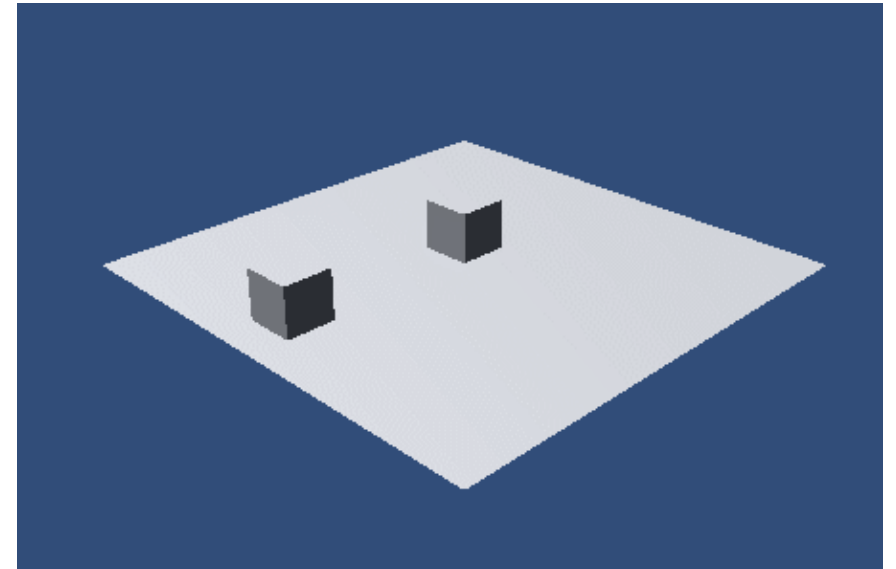
- Physic distinguishes between

- **Linear dynamics:** moving a body within the space

- **Angular dynamics:** rotation of a body

- **Center of mass:** idealized single point which provides the anchor for physical calculations

- **Force:** acceleration and deceleration of an object,
depending on its mass: $F = m * a$

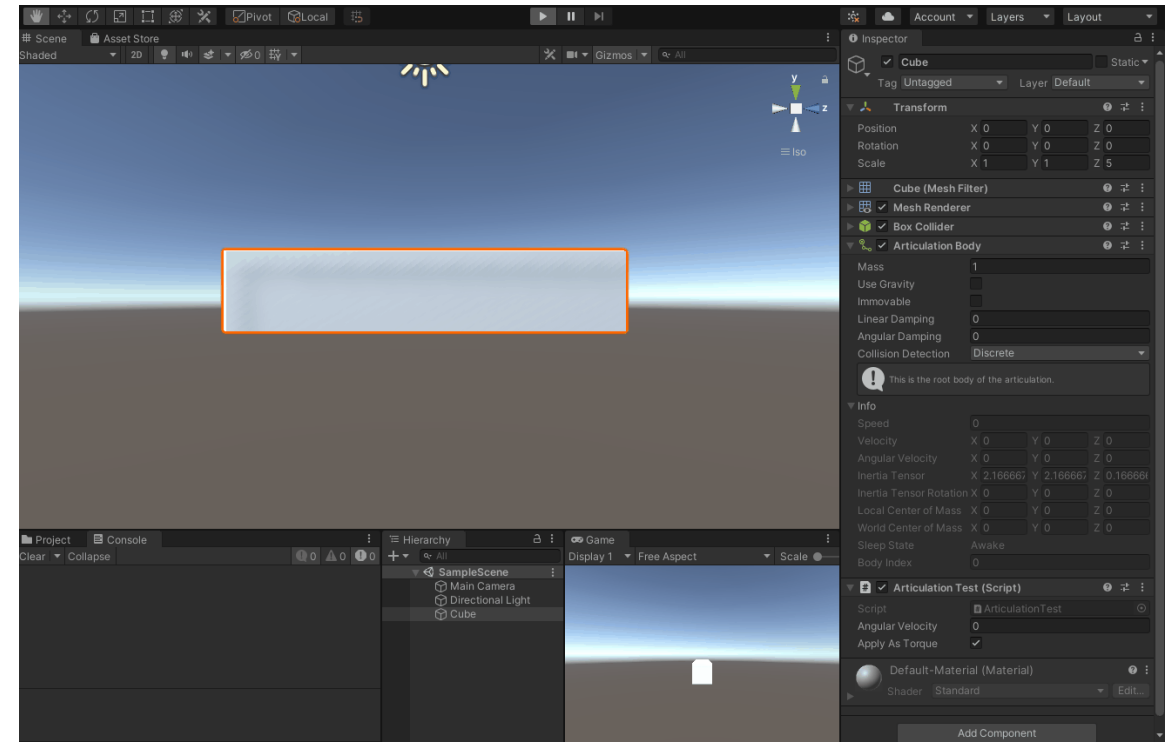


Game Loop

Physics

→ Rigid Body Angular Dynamics

- **Rotation** around an axis (radians rad or degrees °)
- **Angular speed:** constant rotation (rad/s, °/s)
- **Angular acceleration:** increasing/decreasing angular speed (rad/s², °/s²)
- **Moment of inertia (I):** While the mass defines the force needed for a linear motion, the moment of inertia defines the force needed to change the angular speed
 - **Mass near the axis:** small moment of inertia
 - **Mass away from the axis:** high moment of inertia
 - **Torque:** measure of force for rotation (i.e., depending on I, more or less torque is needed)

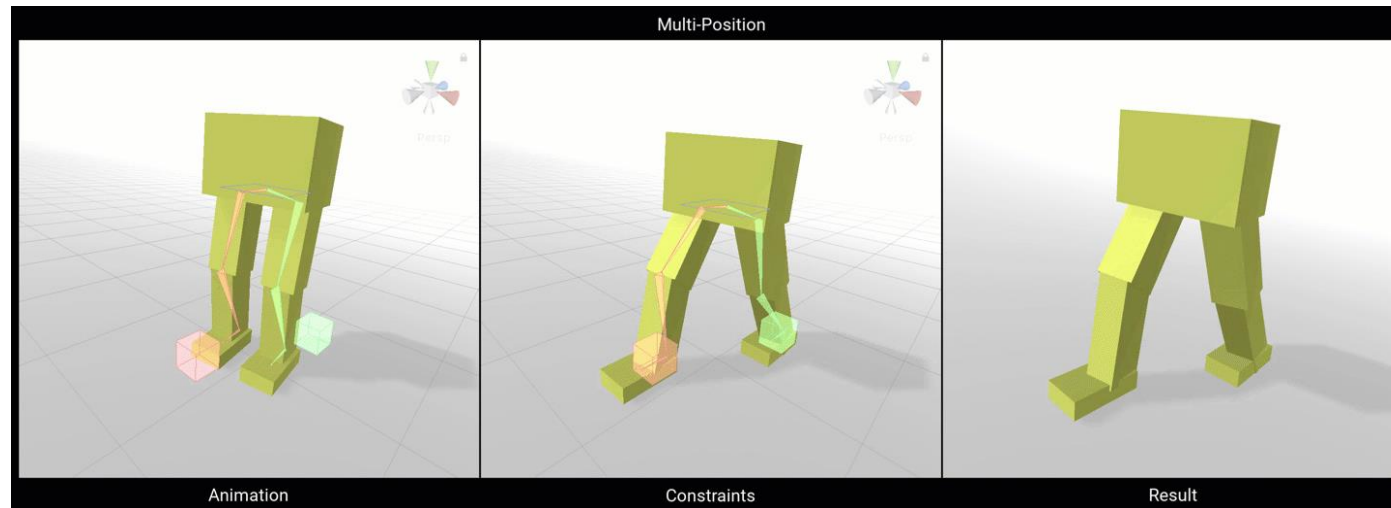


Game Loop

Physics

→ Constraints

- **Point constraints:** Two points connect two objects as if they would be unified, physics applies to the compound object
- **Stiff spring:** point constraint over a distance
- **Hinge constraint:** limits rotation to only a hinge axis (i.e., for a door)
- **Prismatic constraint:** limits motion to a single degree of freedom (i.e., for a character moving 2D in a 3D-world)
- **Constraint chains:** collection of multiple constraints
- **Rag Doll:** constraint chain of a body with multiple “limbs”



Game Loop

Physics

→ Design Impacts

- **Predictability:** Animations and clips are more predictable
- **Control:** Including physics makes the game less controllable for the programmer
- **Unexpectedness:** some things may just happen without anyone knowing before...

→ Engineering Impacts

- **UI:** How to control objects in the physical environment?
- **AI:** Actions and paths must be dynamic, as objects may block it anytime
- **Graphics:** Glitches, objects getting stuck in each other, etc.
- **Multiplayer:** Where is the physics simulated (client vs. server)?
- **Debugging:** Record/playback may not work properly (i.e., no direct comparison)

Game Engine Concepts



UNIVERSITY
OF APPLIED SCIENCES
UPPER AUSTRIA

FH Oberösterreich

University of Applied Sciences Upper Austria

Digital Media Department

Media Technology & Design

Digital Arts

Interactive Media



Dr. Andreas Riegler

Professor of Interactive Systems

Digital Media Department

E Andreas.Riegler@fh-hagenberg.at